

ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅՈՒՆ ԿՐԹՈՒԹՅԱՆ ԵՎ ԳԻՏՈՒԹՅԱՆ
ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ
ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ
ՏՀՏԷ ԻՆՍՏԻՏՈՒՏ ՏԵՂԵԿԱՏՎՈՒԹՅԱՆ ԱՆՎՏԱՆԳՈՒԹՅԱՆ ԵՎ ԾՐԱԳՐԱՅԻՆ
ԱՊԱՀՈՎՄԱՆ ԱՄԲԻՈՆ



Լաբորատոր աշխատանք 10

Ինստիտուտ՝ ՏՀՏԷ
Առարկա՝ Ծրագրային ապահովման անվտանգություն
Թեմա՝ Ծրագրային ապահովման պաշտպանություն՝
խեղաթյուրման (Obfuscation) ալգորիթմներով
Խումբ՝ SS-355
Ուսանող՝ Մելքոնյան Սոնա

Երևան 2026

control_flow.py

hidden_multiply(a, b) ֆունկցիան կատարում է բազմապատկում control flow obfuscation մեթոդով:
Գործողությունը թաքցված է while True ցիկլի և state փոփոխականի մեջ, ինչը բարդացնում է reverse engineering-ը:

AdvancedObfuscation.cpp

Ֆայլում օգտագործվում են advanced obfuscation մեթոդներ:

- ADD(a,b) macro-ն փոխարինում է սովորական գումարումը բիթային գործողություններով:
- template Add-ը հաշվարկը կատարում է compile-time-ում:
- inline ASM հատվածում օգտագործվում է assembly հրաման, որը բարդացնում է կոդի վերլուծությունը:

controlflow.cpp

obfuscated_sum(a, b) ֆունկցիան օգտագործում է dead code և control flow obfuscation:

- for ցիկլը useless code է և արդյունքի վրա չի ազդում:
- while(true) + state կառուցվածքը թաքցնում է իրական կատարման հոսքը:

main.py

main.py ֆայլը միավորում է XOR encoding-ը և control flow obfuscation-ը:

- "Secret" տեքստը encode է արվում XOR մեթոդով:
- decode-ի միջոցով վերականգնվում է սկզբնական տեքստը:
- hidden_multiply(3,4) ֆունկցիան վերադարձնում է 12:

obfuscation.py

Ֆայլում օգտագործվում են XOR encoding և control flow obfuscation մեթոդներ:

- xor_encode() ֆունկցիան տեքստը վերածում է թվերի զանգվածի:
- xor_decode() վերականգնում է սկզբնական տեքստը:
- obfuscated_sum() ֆունկցիան օգտագործում է dead code և state-based execution flow:

```
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/Լաբ10$ python3 obfuscation.py
Encoded: [95, 114, 123, 123, 120, 55, 64, 120, 101, 123, 115]
Decoded: Hello World
12
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/Լաբ10$ python3 xor_obfuscation
.py
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/Լաբ10$ python3 control_flow.py
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/Լաբ10$ python3 main.py
Encoded: [95, 105, 111, 126, 105, 120]
Decoded: Secret
Multiply: 12
```

```
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/Լաբ10$ ./xor.exe
Encoded: 95 114 123 123 120 55 64 120 101 123 115
Decoded: Hello World
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/Լաբ10$ g++ controlflow.cpp -o controlflow.e
xe
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/Լաբ10$ ./controlflow.exe
Result: 12
```

Լրացուցիչ աշխատանք

```
polymorphic_demo.cpp > main()
1  #include <iostream>
2  #include <cstdlib>
3  #include <ctime>
4  using namespace std;
5
6  int main() {
7      srand(time(0));
8
9      int a = 10;
10     int b = 5;
11     int result;
12
13     int variant = rand() % 3;
14
15     if (variant == 0) {
16         result = a + b;
17         cout << "Variant 1 executed\n";
18     }
19     else if (variant == 1) {
20         result = (a ^ b) + 2 * (a & b);
21         cout << "Variant 2 executed\n";
22     }
23     else {
24         result = b + a;
25         cout << "Variant 3 executed\n";
26     }
27
28     cout << "Result = " << result << endl;
29     return 0;
30 }
```

Նույն գործողությունը՝ $a + b$, կատարվում է տարբեր ձևերով: Ամեն գործարկման ժամանակ ծրագիրը կարող է ընտրել տարբեր տարբերակ, բայց արդյունքը մնում է նույնը:
g++ polymorphic_demo.cpp -o polymorphic_demo.exe

```
polymorphic_demo.exe
1  Variant 2 executed
2
3  Variant 3 executed
4
5  Result = 15
```

IsDebuggerPresent() ֆունկցիան, որը ստուգում է՝ ծրագիրը debugger-ի տակ աշխատում է, թե ոչ:

```
anti_debugging.cpp > main()
1  #include <iostream>
2  #include <windows.h>
3  using namespace std;
4
5  int main() {
6      if (IsDebuggerPresent()) {
7          cout << "Debugger detected!" << endl;
8          return 1;
9      }
10
11     cout << "No debugger detected. Program is running normally." << endl;
12     return 0;
13 }
```

սարգ password checker, որը կարելի է բացել Ghidra-ով կամ IDA-ով և վերլուծել main ֆունկցիան, strings-ը և control flow-ը

```
crackme.cpp > main()
1  #include <iostream>
3  using namespace std;
4
5  int main() {
6      string password;
7
8      cout << "Enter password: ";
9      cin >> password;
10
11     if (password == "secret123") {
12         cout << "Access granted!" << endl;
13     } else {
14         cout << "Access denied!" << endl;
15     }
16
17     return 0;
18 }
```